

Trabalho 1 Bimestre, 3 Pontos

Data de entrega: 10 de setembro (início da aula) – não será aceito após esse horário

Valor: 3,0 pontos

Forma de apresentação: Sorteio de duplas para demonstração ao vivo em sala de aula (o projeto deve rodar perfeitamente na máquina do aluno ou do professor).

1. OBJETIVO GERAL

Desenvolver uma aplicação web simples (CRUD – Create, Read, Update, Delete) utilizando PHP e um banco de dados relacional (MySQL, MariaDB, PostgreSQL, etc. – a escolha fica a critério da dupla), e orquestrar todo o ambiente com Docker Compose. O trabalho deve ser versionado no GitHub (repositório público) e acompanhado de documentação clara no README.md. O foco é demonstrar a capacidade de containerizar uma aplicação completa, configurar a comunicação entre serviços e seguir boas práticas de organização e versionamento.

2. REQUISITOS FUNCIONAIS (O CRUD)

A aplicação PHP deve implementar as quatro operações básicas sobre uma entidade de sua escolha (ex.: Produtos, Alunos, Tarefas, etc.). A entidade deve conter pelo menos 3 campos (além do ID), por exemplo:

- id (chave primária, auto incremento)
- nome (varchar)
- descricao (texto)
- data_cadastro (date/datetime)

A aplicação deve conter:

- Página de listagem de todos os registros.
- Página com formulário de cadastro (envio via POST).
- Página com formulário de edição (pré-carregado com os dados do registro).
- Exclusão de registros (pode ser via link com confirmação ou diretamente).

3. REQUISITOS DE INFRAESTRUTURA (DOCKER COMPOSE)

3.1. Escolha das imagens

- Para a aplicação PHP, utilize uma imagem pronta do Docker Hub.
- Para o banco de dados, utilize a imagem oficial do SGBD escolhido (MySQL, MariaDB, PostgreSQL, etc.). Não há versão obrigatória – a versão fica a critério da dupla.

3.2. docker-compose.yml

O arquivo docker-compose.yml deve orquestrar pelo menos dois serviços:

- app: container que executa a aplicação PHP.
- db: container do banco de dados.

Além disso, o compose deve conter:

- Variáveis de ambiente para configurar a conexão com o banco diretamente no serviço app (ex.: DB_HOST, DB_USER, DB_PASSWORD, DB_NAME). Use a seção environment do docker-compose – não é permitido o uso de arquivo .env.
- Mapeamento de porta para acessar a aplicação no navegador (ex.: 8080:80 ou porta equivalente).
- Volume persistente para o banco de dados, garantindo que os dados não sejam perdidos ao reiniciar os containers.
- Rede personalizada (bridge) para que os serviços se comuniquem pelos nomes dos containers.

3.3. Comentários no docker-compose.yml

Você deverá comentar cada linha do arquivo docker-compose.yml explicando o que cada diretiva faz. Os comentários devem ser inseridos diretamente no arquivo, em português, de forma didática. Exemplo:

services:

app:

image: php:apache # Utiliza a imagem oficial do PHP com Apache embutido

container_name: meu-app # Define um nome fixo para o container

ports:

- "8080:80" # Mapeia a porta 80 do container para a porta 8080 do host

environment: # Variáveis de ambiente usadas pelo código PHP

DB_HOST: db # Nome do serviço do banco de dados (resolvido pela rede interna)

DB_USER: root

DB_PASSWORD: root

DB_NAME: crud

networks:

- minha-rede # Conecta este container à rede personalizada

Essa explicação detalhada será avaliada como parte da documentação.

4. REQUISITOS DE VERSIONAMENTO (GITHUB)

- O repositório deve ser público (para permitir acesso do professor e da turma).
- Os commits devem ser frequentes e com mensagens claras (ex.: "feat: adiciona formulário de cadastro", "fix: corrige conexão com o banco").
- A branch principal deve ser main (ou master) e deve conter a versão final e funcional do projeto.

5. REQUISITOS DO README.md

O arquivo README.md deve ser completo e bem estruturado, contendo obrigatoriamente os seguintes tópicos:

1. Título e descrição do projeto (o que faz, qual entidade foi escolhida).
2. Pré-requisitos (Docker e Docker Compose instalados).
3. Passo a passo para executar o projeto:
 - Clonar o repositório.
 - Subir os containers com `docker-compose up -d`.
 - Explicar como a tabela do banco é criada (por exemplo, via código PHP que verifica e cria automaticamente, ou via comando manual com um script SQL – se for manual, descreva o comando).
 - Acessar a aplicação em `http://localhost:8080` (ou porta configurada).
4. Explicação detalhada do `docker-compose.yml` – além dos comentários no arquivo, no README você deve descrever, em texto, a função de cada serviço, as variáveis de ambiente utilizadas e a rede criada.
5. Pontos interessantes observados pela dupla – liste, no mínimo, três aprendizados ou decisões técnicas que vocês consideram relevantes. Exemplos:
 - "Utilizamos variáveis de ambiente diretamente no compose, o que facilita a mudança de configuração sem alterar o código."
 - "Configuramos um volume para o banco, garantindo a persistência dos dados."
 - "Criamos uma rede isolada para a comunicação entre os containers, aumentando a segurança."
 - "Optamos por uma imagem PHP pronta que já incluía a extensão necessária, simplificando o setup."
6. Autores (nomes completos da dupla).

6. RESTRIÇÕES E OBSERVAÇÕES IMPORTANTES

- Não é permitido o uso de arquivo `.env` para este trabalho.

- Não é permitido o uso de healthcheck, scripts de inicialização automática ou ferramentas de logging avançadas (isso foge do escopo atual).
- Você pode escolher qualquer SGBD (MySQL, MariaDB, PostgreSQL, etc.) e qualquer versão, desde que funcione com PHP.
- Todo o desenvolvimento deve ser feito antes da aula do dia 10/09 – o trabalho não poderá ser finalizado em sala de aula.
- A apresentação será definida por sorteio; portanto, testem exaustivamente o projeto em uma máquina limpa (que não tenha PHP ou banco de dados instalados localmente) para garantir que ele rode apenas com Docker.

7. ENTREGA E APRESENTAÇÃO

- Entrega: No início da aula do dia 10/09, a dupla deve informar o link do repositório público ao professor. Não serão aceitos repositórios privados ou envios por e-mail.
- Apresentação: Após a entrega, o professor sorteará algumas duplas para demonstrar o funcionamento do projeto ao vivo. A dupla sorteada deve:
 1. Clonar o repositório em uma máquina com Docker instalado.
 2. Executar os comandos descritos no README.
 3. Navegar pela aplicação, mostrando as quatro operações do CRUD.
 4. Explicar brevemente a estrutura do docker-compose.yml e comentar as decisões tomadas.